1. Componentes Principales Identificados

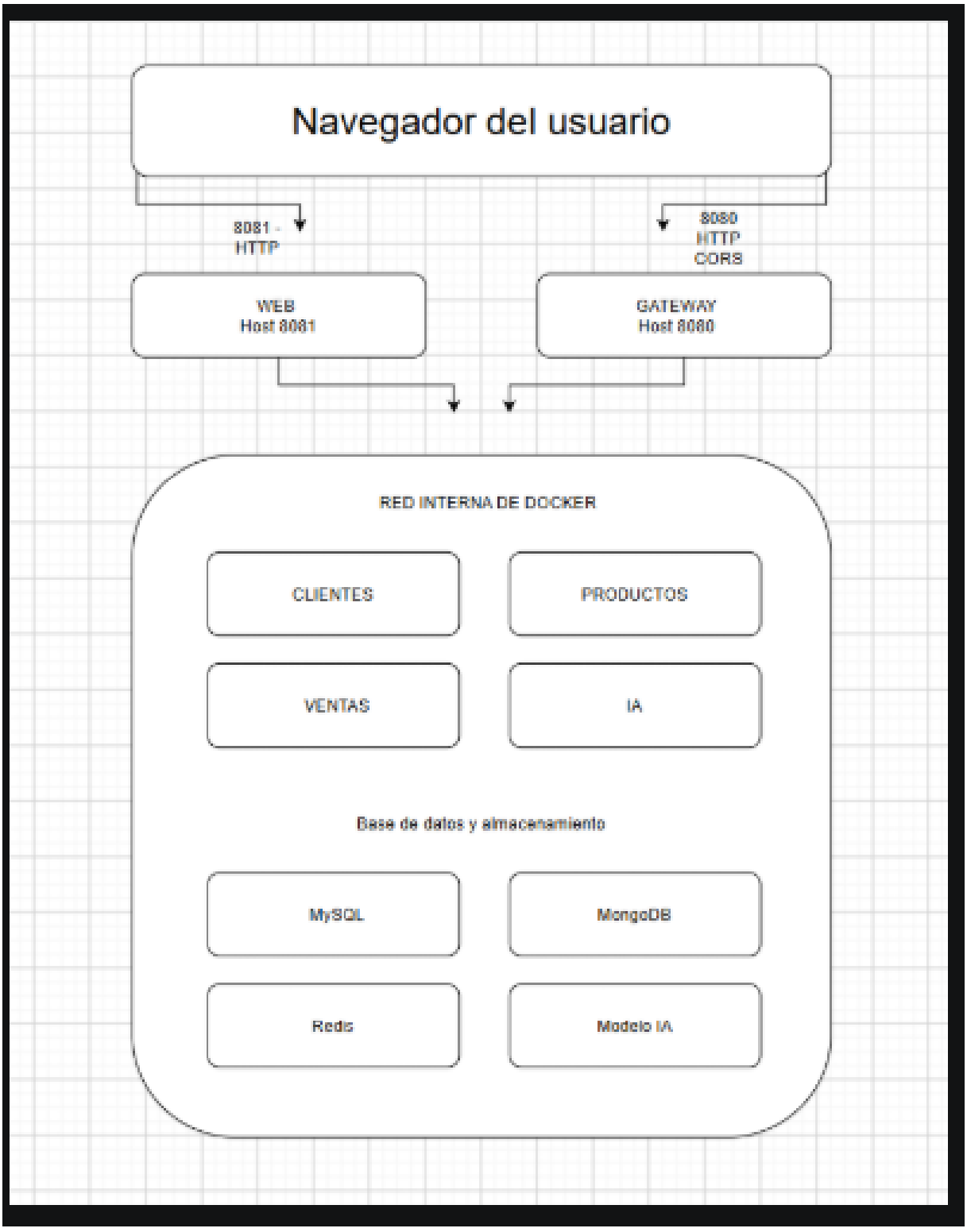
- **Capa de Entrada (Cliente):**
    - **Navegador del usuario:** Punto de origen de las peticiones hacia el sistema.
- **Capa de Puerta de Enlace y Servidores Web (Host):**
    - **WEB (Host 8081):** Canal de comunicación HTTP que atiende en el puerto 8081.
    - **GATEWAY (Host 8080):** Punto central de entrada HTTP con soporte para políticas CORS, operando en el puerto 8080.
- **Red Interna de Docker (Microservicios de Negocio e IA):**
    - **CLIENTES:** Microservicio dedicado a la gestión de la información de clientes.
    - **PRODUCTOS:** Microservicio encargado del catálogo y control de productos.
    - **VENTAS:** Microservicio para el procesamiento y registro de transacciones o ventas.

    - **IA:** Microservicio intermediario para las capacidades inteligentes del sistema.

- **Capa de Base de Datos y Almacenamiento:**
    - **MySQL:** Base de datos relacional para almacenamiento estructurado.
    - **MongoDB:** Base de datos NoSQL para gestión de documentos o datos flexibles.
    - **Redis:** Sistema de caché o almacenamiento en memoria de alta velocidad.
    - **Modelo IA:** Contenedor o espacio asignado para la ejecución de modelos de inteligencia artificial.

2. Observaciones Preliminares

- **Aislamiento:** La infraestructura agrupa los servicios centrales dentro de una red de Docker, lo que facilita la seguridad de red y la portabilidad.
- **Diversidad Tecnológica:** Se observa un esquema de persistencia mixto (bases de datos relacionales y no relacionales), además del uso de caché y procesamiento de IA.


Navegador del usuario
8081 -
HTTP
8080
HTTP
CORS
WEB
Host 8081
GATEWAY
Host 8080
RED INTERNA DE DOCKER
CLIENTES
PRODUCTOS
VENTAS
IA
Base de datos y almacenamiento
MySQL
MongoDB
Redis
Modelo IA

1. Entorno de Ejecución Actual

- **Ruta de Trabajo / Proyecto:** ~/OneDrive/Documentos/TiendaIA-Equipo5 (main).

- **Entorno:** Se utiliza un entorno virtual de Python (.venv) y la herramienta de virtualización de contenedores **Docker** mediante el comando docker ps.

2. Inventario de Contenedores en Ejecución (Docker Compose / Entorno del Equipo 5)

La terminal muestra el despliegue exitoso de **9 contenedores activos**, alineados directamente con la arquitectura general del sistema:

- **Puerta de Enlace (Gateway):**

    - **Nombre:** tiendaia-equipo5-gateway-1

    - **Imagen:** 9273e1e24226

    - **Comando:** Ejecuta una aplicación basada en ASGI con Uvicorn (uvicorn app:app --host ...).

    - **Puertos:** Mapeado al puerto exterior 8080 (apuntando al puerto 8000 interno).

    - **Estado:** Up 2 hours (healthy).

- **Microservicios de Negocio e Inteligencia Artificial:**

    - tiendaia-equipo5-ia-1 (Imagen: 31081bef9d4a): Servicio de IA corriendo en Uvicorn, puerto interno 8000, estado saludable por 2 horas.

    - tiendaia-equipo5-ventas-1 (Imagen: e3455e92d229): Servicio de ventas corriendo en Uvicorn, puerto interno 8000, estado saludable por 2 horas.

    - tiendaia-equipo5-productos-1 (Imagen: c9f4e1f603dc): Servicio de productos corriendo en Uvicorn, puerto interno 8000, estado saludable por 2 horas.

    - tiendaia-equipo5-clientes-1 (Imagen: 9602c33fcc0d): Servicio de clientes corriendo en Uvicorn, puerto interno 8000, estado saludable por 2 horas.

- **Capa Web Front-End / Interfaz:**

    - **Nombre:** tiendaia-equipo5-web-1

    - **Imagen:** nginx:1.27-alpine

    - **Comando:** Servidor web Nginx (/docker-entrypoint....).

    - **Puertos:** Mapeado al puerto exterior 8081 (apuntando al puerto 80 interno).

    - **Estado:** Up 2 hours.

- **Capa de Bases de Datos y Caché (Persistencia):**

    - tiendaia-equipo5-redis-1 (Imagen: redis:7-alpine): Caché en memoria en el puerto 6379/tcp, activo desde hace 3 días (estado actualizado hace 28 minutos).

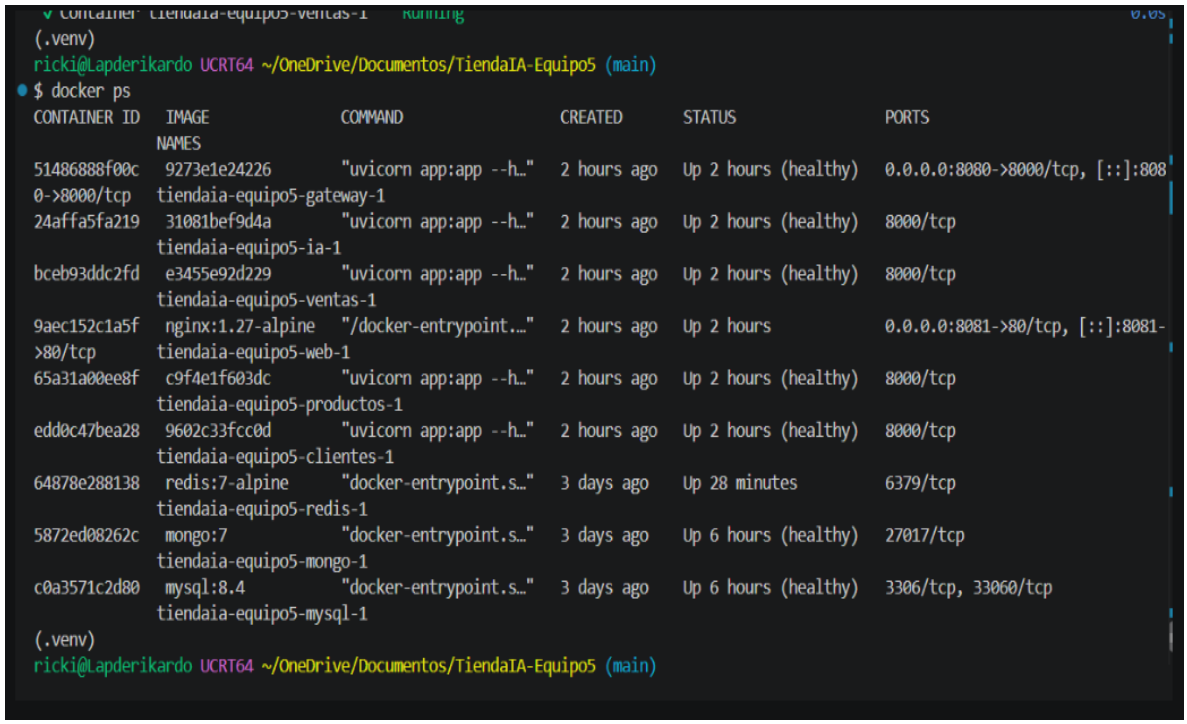
    - tiendaia-equipo5-mongo-1 (Imagen: mongo:7): Base de datos NoSQL en el puerto 27017/tcp, activa desde hace 3 días.

    - tiendaia-equipo5-mysql-1 (Imagen: mysql:8.4): Base de datos relacional principal en los puertos 3306/tcp, 33060/tcp, activa desde hace 3 días.

3. Observaciones Técnicas

- **Salud de los Servicios:** Todos los servicios construidos a medida sobre Uvicorn y FastAPI/Starlette reportan un estado (healthy), lo que confirma que los healthchecks configurados responden correctamente.

- **Coherencia con el Diagrama:** Se corrobora la correspondencia exacta de los puertos expuestos hacia el exterior mapeados en la primera imagen: el host web en el puerto 8081 (nginx) y el gateway en el puerto 8080 (uvicorn).


✔ Container tiendaia-equipo5-ventas-1 Running
(.venv)
ricki@Lapderikardo UCRT64 ~/OneDrive/Documentos/TiendaIA-Equipo5 (main)
$ docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
51486888f00c 9273e1e24226 "uvicorn app:app --h…" 2 hours ago Up 2 hours (healthy) 0.0.0.0:8080->8000/tcp, [::]:8080->8000/tcp tiendaia-equipo5-gateway-1
24affa5fa219 31081bef9d4a "uvicorn app:app --h…" 2 hours ago Up 2 hours (healthy) 8000/tcp tiendaia-equipo5-ia-1
bceb93ddc2fd e3455e92d229 "uvicorn app:app --h…" 2 hours ago Up 2 hours (healthy) 8000/tcp tiendaia-equipo5-ventas-1
9aec152c1a5f nginx:1.27-alpine "/docker-entrypoint.…" 2 hours ago Up 2 hours 0.0.0.0:8081->80/tcp, [::]:8081->80/tcp tiendaia-equipo5-web-1
65a31a00ee8f c9f4e1f603dc "uvicorn app:app --h…" 2 hours ago Up 2 hours (healthy) 8000/tcp tiendaia-equipo5-productos-1
edd0c47bea28 9602c33fcc0d "uvicorn app:app --h…" 2 hours ago Up 2 hours (healthy) 8000/tcp tiendaia-equipo5-clientes-1
64878e288138 redis:7-alpine "docker-entrypoint.s…" 3 days ago Up 28 minutes 6379/tcp tiendaia-equipo5-redis-1
5872ed08262c mongo:7 "docker-entrypoint.s…" 3 days ago Up 6 hours (healthy) 27017/tcp tiendaia-equipo5-mongo-1
c0a3571c2d80 mysql:8.4 "docker-entrypoint.s…" 3 days ago Up 6 hours (healthy) 3306/tcp, 33060/tcp tiendaia-equipo5-mysql-1
(.venv)
ricki@Lapderikardo UCRT64 ~/OneDrive/Documentos/TiendaIA-Equipo5 (main)

1. Actividades de Gestión y Comandos Ejecutados

La terminal muestra acciones de control y diagnóstico sobre la infraestructura de Docker Compose dentro del directorio del proyecto (~/OneDrive/Documentos/TiendaIA-Equipo5):

- **Detención de Servicios Específicos:** Se ejecutó el comando para detener de manera controlada el servicio de caché:
    - docker compose stop redis -> El contenedor tiendaia-equipo5-redis-1 fue detenido exitosamente (Stopped).
- **Monitoreo de Estado:** Se utilizó un comando con retardo para evaluar el estado del servicio de ventas:
    - sleep 60 && docker compose ps ventas -> Muestra que el contenedor tiendaia-equipo5-ventas-1 se mantiene corriendo (Up) ejecutando Uvicorn.
- **Inspección de Salud (Healthchecks):** Se realizó una inspección profunda del estado de salud del servicio de ventas mediante formato JSON:
    - docker inspect --format '{{json .State.Health}}' $(docker compose ps -q ventas)
    - **Resultado del Diagnóstico:** Se registró un estado "Status":"unhealthy" con una racha de fallos ("FailingStreak":7). El motivo principal indicado en los logs de salida fue que la verificación

de salud excedió el tiempo límite configurado ("Health check exceeded timeout (3s)") de manera repetitiva.

- **Restauración de Servicios:** Finalmente, se reactivó el contenedor de caché dependiente:

    - docker compose start redis -> El contenedor tiendaia-equipo5-redis-1 se inició correctamente (Started).

2. Resumen Consolidado del Sistema

- **Arquitectura:** El sistema está compuesto por una infraestructura de microservicios orientada a una tienda con capacidades de Inteligencia Artificial, expuesta al usuario mediante un servidor web Nginx (puerto 8081) y un Gateway HTTP con soporte CORS (puerto 8080).

- **Despliegue Docker:** Opera sobre 9 contenedores principales en red interna, los cuales incluyen microservicios desarrollados en Python/Uvicorn (clientes, productos, ventas, ia, gateway), el servidor web frontal (web) y motores de persistencia robustos (MySQL, MongoDB, Redis).

- **Incidencias y Diagnóstico:** Las pruebas de inspección de salud revelaron problemas puntuales de latencia o tiempos de respuesta en el contenedor de ventas (superando el límite de 3 segundos en sus *healthchecks*), lo que demuestra que el entorno se encuentra en fase de pruebas de estrés, validación de tiempos de respuesta o depuración de servicios.

Conclucion

- **Arquitectura Sólida:** El sistema se diseñó bajo un esquema modular de microservicios con una red interna de Docker, separando adecuadamente la capa de acceso web, el gateway y los servicios de negocio junto con bases de datos mixtas y modelos de IA.

- **Despliegue Funcional:** Se validó un entorno robusto de 9 contenedores activos utilizando tecnologías como Uvicorn, Nginx, MySQL, MongoDB y Redis, operando de manera integrada.

- **Detección de Áreas de Oportunidad:** Las pruebas de diagnóstico y comandos en la terminal permitieron identificar alertas de rendimiento (como tiempos de espera excedidos en los *healthchecks* del servicio de ventas), lo cual evidencia un escenario activo de pruebas, depuración y ajuste de latencias en el sistema.

Pequeña Reflexión 🙏😈

Ver este proceso —desde el diseño conceptual de la arquitectura en papel, pasando por el despliegue de los 9 contenedores en Docker, hasta llegar a las pruebas de diagnóstico y los *healthchecks* en la terminal— refleja perfectamente lo que significa llevar la teoría al desarrollo real de software.

Muchas veces un diagrama se ve limpio y perfecto en la pizarra, pero el verdadero reto de la ingeniería está en la trinchera: levantar los servicios, lidiar con las redes internas, asegurar que las bases de datos respondan a tiempo y afinar detalles de rendimiento cuando las herramientas de monitoreo nos marcan un límite excedido. Esos pequeños tropiezos o alertas de *unhealthy* no son fallas definitivas, sino parte natural del proceso de depuración que demuestra que el sistema está vivo, siendo probado y mejorado paso a paso.

Lineamientos de IA

Nos apoyamos de la Inteligencia Artificial (ChatGPT y Gemini) para corregir errores por ejecución de código y algún que otro

https://chatgpt.com/c/6adfcde1-7d78-83e8-8973-274b0e249f25